



МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА – Российский технологический университет»
РТУ МИРЭА

Институт информационных технологий
Кафедра прикладной математики

РАБОТА ДОПУЩЕНА К ЗАЩИТЕ

И.о. заведующего
кафедрой _____ Т.Е. Смоленцева

«01» _____ июня 2024 г.

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА
по направлению подготовки бакалавров 01.03.04 Прикладная математика

На тему: Разработка нейронной сети портретной классификации

Обучающийся

подпись

Черных Анастасия Николаевна

Фамилия, имя, отчество

шифр 20И0379

группа ИМБО-01-20

Руководитель работы

подпись

к.т.н.,
доцент

Ледовская Е.В.

Москва 2024 г.



МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА – Российский технологический университет»
РТУ МИРЭА

Институт информационных технологий

Кафедра прикладной математики

СОГЛАСОВАНО

И.о. заведующего
кафедрой

подпись

Смоленцева Татьяна Евгеньевна

«14»

февраля

2024 г.

УТВЕРЖДАЮ

Директор
института

подпись

Зуев Андрей Сергеевич

«14»

февраля

2024 г.

ЗАДАНИЕ

на выполнение выпускной квалификационной работы бакалавра

Обучающийся

Черных Анастасия Николаевна

Фамилия Имя Отчество

Шифр

20И0379

Направление
подготовки

01.03.04

индекс направления

Прикладная математика

наименование направления

Группа

ИМБО-01-20

1. Тема выпускной квалификационной работы: Разработка нейронной сети портретной классификации

2. Цель и задачи выпускной квалификационной работы

Цель работы: Разработка нейронной сети портретной классификации в условиях применения новых технических и технологических средств сбора, передачи, обработки и выдачи информации

Задачи работы: характеристика предметной области, описание существующей технологии выполнения выбранной функции управления, постановка задачи на разработку НС; обзор существующих разработок; экономические параметры разработки и внедрения НС; обоснование выбора средств разработки, описание реализации НС, описание технологии работы с информационной системой

3. Этапы выпускной квалификационной работы

№ этапа	Содержание этапа выпускной квалификационной работы	Результат выполнения этапа ВКР	Срок выполнения
1	Исследовательский раздел	<i>Выполнено</i> <i>Евг</i>	21.03.2024
1.1	Характеристика предметной области		
1.1.1	Характеристика объекта исследования		
1.1.2	Характеристика предмета исследования		
1.2	Обзор существующих решений		
1.3	Формулировка исследовательской задачи подхода к разработке нейронной сети		
2	Аналитический раздел	<i>Выполнено</i> <i>Евг</i>	18.04.2024
2.1	Анализ и характеристика входных данных		
2.1.1	Изображения человеческих лиц		
2.1.2	Изображения рисованных лиц		
2.2	Описание этапов разработки модели		
2.3	Обоснование выбора инструментальных средств разработки		
2.3.1	Обоснование выбора языка программирования Python		
2.3.2	Обоснование выбора библиотеки OpenCV		
2.3.3	Обоснование выбора сервиса FNM для нормализации изображений		
2.3.4	Обоснование выбора библиотеки Tensorflow		
3	Технологический раздел	<i>Выполнено</i> <i>Евг</i>	16.05.2024
3.1	Описание реализации разработанной нейронной сети		
3.1.1	Сбор и предобработка входных данных		
3.1.2	Нормализация изображений		
3.1.3	Обучение классификатора		
3.1.4	Классификация изображений		
3.2	Анализ полученных результатов		
4	Введение, заключение, список источников, приложения	<i>Выполнено</i> <i>Евг</i>	17.05.2024
5	Презентационный материал, аннотация	<i>Выполнено</i> <i>Евг</i>	24.05.2024
6	Нормоконтроль	<i>Выполнено</i> <i>Евг</i>	25.05.2024

4. Перечень разрабатываемых документов и графических материалов: электронная версия выпускной квалификационной работы бакалавра, презентационный материал с основными результатами выпускной квалификационной работы бакалавра – графический материал по исследованию предметной области

5. Руководитель и консультанты выпускной квалификационной работы

Функциональные обязанности	Должность в Университете	Фамилия, имя, отчество	Подпись
Руководитель ВКР	доцент	Ледовская Екатерина Валерьевна	<i>Евг</i>

Задание выдал
Руководитель ВКР: *Евг*
подпись

«14» февраля 2024 г.

Задание принял к исполнению
Обучающийся: *М*
подпись

«14» февраля 2024 г.

АННОТАЦИЯ

Исполнитель: студент группы ИМБО-01-20 Черных А.Н.

Руководитель: доцент кафедры ПМ, к.т.н. Ледовская Е.В.

Выпускная квалификационная работа на тему «Разработка нейронной сети портретной классификации».

Общее количество страниц: 65.

Работа включает в себя 39 рисунков, 1 таблицу, 5 приложений.

Список источников информации включает 43 позиции.

Ключевые слова: анализ изображений лиц, CLIP, машинное обучение, классификация.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	5
1 ИССЛЕДОВАТЕЛЬСКИЙ РАЗДЕЛ	7
1.1 Характеристика предметной области.....	7
1.1.1 Характеристика объекта исследования	9
1.1.2 Характеристика предмета исследования.....	13
1.2 Обзор существующих решений.....	17
1.3 Формулировка исследовательской задачи подхода к разработке нейронной сети.....	22
2 АНАЛИТИЧЕСКИЙ РАЗДЕЛ.....	23
2.1 Анализ и характеристика входных данных	23
2.1.1 Изображения человеческих лиц	24
2.1.2 Изображения рисованных лиц.....	26
2.2 Описание этапов разработки модели	28
2.3 Обоснование выбора инструментальных средств разработки.....	29
2.3.1 Обоснование выбора языка программирования Python	29
2.3.2 Обоснование выбора библиотеки OpenCV	30
2.3.3 Обоснование выбора сервиса FNM для нормализации изображений	32
2.3.4 Обоснование выбора библиотеки Tensorflow	33
3 ТЕХНОЛОГИЧЕСКИЙ РАЗДЕЛ	34
3.1 Описание реализации разработанной нейронной сети	34
3.1.1 Сбор и предобработка входных данных	35
3.1.2 Нормализация изображений	36
3.1.3 Обучение классификатора	39
3.1.3 Классификация изображений	40
3.2 Анализ полученных результатов	41
ЗАКЛЮЧЕНИЕ	43
СПИСОК ИСТОЧНИКОВ	45
ПРИЛОЖЕНИЯ.....	50

ВВЕДЕНИЕ

За последние несколько лет было совершено большое количество прорывов в сфере компьютерного зрения, благодаря чему создано множество механизмов, позволяющих работать с изображениями лиц. Применение новейших методов компьютерного зрения позволяет не только точно идентифицировать личность людей на фотографиях и видео, но и анализировать различные характеристики, такие как эмоции, возраст, пол и другие. Дальнейшее развитие и совершенствование методов компьютерного зрения для работы с изображениями лиц будет способствовать появлению еще более эффективных и функциональных технологий, которые в перспективе смогут кардинально изменить многие аспекты нашей жизни.

Однако в области анализа изображений лиц можно выделить актуальную проблему точности извлечения лиц. Она до сих пор остро стоит, так как различие между рисованными и реальными лицами не так просто определяется в процессе извлечения их из изображений. Этому способствует уникальная природа схожести лиц людей и различных выдуманных героев. Применение лиц персонажей за лица реальных людей может сильно испортить результаты многих исследований и привести к избыточности данных. Исходя из этого, классификация лиц на рисованные и реальные представляет собой важное направление исследований, поскольку имеет широкий спектр приложений. Она может быть задействована во всех областях, включающих в себя анализ изображений лиц, начиная от безопасности и аутентификации до создания инновационных систем распознавания образов.

Цель исследования — разработка метода классификации, различающего рисованные и реальные изображения лиц. Исходя из этого, объектом исследования является классификация, а предметом — анализ изображений лиц. Задачи, необходимые для выполнения в ходе исследования:

1. Охарактеризовать предметную область.

2. Описать существующие технологии для работы с изображениями лиц.
3. Провести обзор существующих решений.
4. Выбрать сопутствующие средства для разработки.
5. Обосновать выбор средств разработки.
6. Описать реализацию программного кода.
7. Подвести итоги разработки.

Стоит отметить, что разработан сопутствующий презентационный материал, который представлен в Приложении А, а в процессе написания выпускной квалификационной работы автор руководствовался следующими нормативными актами:

1. «О защите населения и территории от чрезвычайных ситуаций природного и техногенного характера» от 21.12.1994 № 68-ФЗ.
2. «Об основах охраны здоровья граждан в Российской Федерации» от 21.11.2011 № 323-ФЗ.
3. «О гражданской обороне» от 12.01.1998 № 28-ФЗ.
4. Приказ Минздравсоцразвития РФ от 04.05.2012 № 477н «Об утверждении перечня состояний, при которых оказывается первая помощь, и перечня мероприятий по оказанию первой помощи».
5. Трудовой кодекс Российской Федерации от 30.12.2001 № 197-ФЗ
6. СанПиН 2.2.2/2.4.1340-03 «Гигиенические требования к персональным электронно-вычислительным машинам и организации работы».

Обратим внимание на то, что разработка позволит добавить недостающее звено в процесс анализа лиц, что откроет новые возможности для дальнейшего совершенствования систем компьютерного зрения и повышения их точности. Об анализе лицевых изображений поговорим подробнее в первой части представленной работы.

1 ИССЛЕДОВАТЕЛЬСКИЙ РАЗДЕЛ

В рамках представленной исследовательской работы рассматривается область анализа изображений лиц. Как и многие другие области анализа данных определённой структуры, данная область включает в себя различные этапы, особенности и инструменты. Для понимания концепции исследования опишем аспекты предметной области подробнее.

1.1 Характеристика предметной области

В первую очередь, определим, что мы понимаем под «анализом изображений лиц». Анализ изображений лиц — это область исследований в компьютерном зрении, которая занимается распознаванием, классификацией и интерпретацией характеристик лиц на изображениях.

Данный анализ широко применяется в рамках:

- обеспечения безопасности (для распознавания лиц людей в аэропортах, общественных местах, банках, доступе к зданиям);
- медицине (для идентификации пациентов, мониторинга состояния здоровья через распознавание выражений лица и других физиологических параметров, которые возможно считать с изображения);
- маркетинге и рекламе (для улучшения персонализации рекламных кампаний, анализа реакций на рекламу и создания точечных решений для потребителей);
- розничной торговле (использование технологий распознавания лиц для анализа поведения покупателей, определения предпочтений и повышения уровня обслуживания);

- электронной коммерции (для обеспечения безопасности транзакций и идентификации пользователей при онлайн-платежах);
- транспортной сфере (использование в системах управления транспортом для контроля доступа к общественному транспорту, повышения безопасности пассажиров и улучшения сервиса, наложения штрафов за безбилетный проезд);
- развлекательной индустрии (для создания более реалистичных и вовлекающих интерактивных сценариев в видеоиграх и в виртуальной реальности);
- социальных сетях (для автоматической идентификации друзей на фотографиях, добавления эффектов и фильтров).

В свою очередь, в рамках анализа изображений в перечисленных областях изначальный объём данных должен пройти несколько этапов обработки. Совокупность этих этапов называется конвейером. [1.1]

Исходя из принципов ETL, простая модель конвейера при обработке данных состоит из их извлечения, непосредственного анализа и преобразования, а также дальнейшей загрузки в необходимом формате в систему, где эти данные будут использоваться. Мы более подробно рассмотрим анализ и преобразование, так как именно эту предметную область охватывает представленное исследование.

В общем случае, процесс преобразования массива изображений можно разделить на следующие этапы:

1. Извлечение массива лиц из массива изображений.
2. Обработка изображений лиц.
3. Анализ лиц (например, классификация по заданной принадлежности, детекция и т.п.) [1.2].

Модель конвейера, включающего перечисленные пункты, представлена на Рисунке 1.1.



Рисунок 1.1 — Модель конвейера для обработки изображений лиц

Рассмотрим подробнее, какие задачи можно решить в рамках перечисленных этапов обработки и какими средствами они могут быть достигнуты.

1.1.1 Характеристика объекта исследования

Как было обозначено выше, модель контейнера обработки лиц состоит из трёх этапов. Рассмотрим данный конвейер подробнее, выявив цели и задачи на каждом этапе.

На первом этапе производится извлечение лица из изображения [1.3] с целью отсечения лишних фрагментов (фрагментов, не включающих в себя непосредственно само лицо). Для этого выполняется несколько глобальных задач:

1. Распознавание области нахождения лица (или нескольких лиц) на изображении.
2. Извлечение найденных областей.
3. Сохранение извлечённых областей в качестве отдельных изображений.

Пример выполнения первой из перечисленных задач можно увидеть на Рисунке 1.2.



Рисунок 1.2 — Выделение области нахождения лица

После извлечения лица из изображения следует провести необходимую обработку полученного результата для возможности дальнейших преобразований [1.4]. Это делается для получения изображений, которые можно будет впоследствии сравнивать между собой.

Опишем кратко механизм работы подобных алгоритмов [1.5]. В традиционных алгоритмах обработки изображений использовалась только интенсивность пикселей, что приводило к высоким вычислительным затратам. Папагеоргиу в своей работе предложил концепцию использования множества признаков, основанных на вейвлетах Хаара. Позже Виола и Джонс модифицировали эту идею и разработали так называемые примитивы Хаара (Рисунок 1.3).

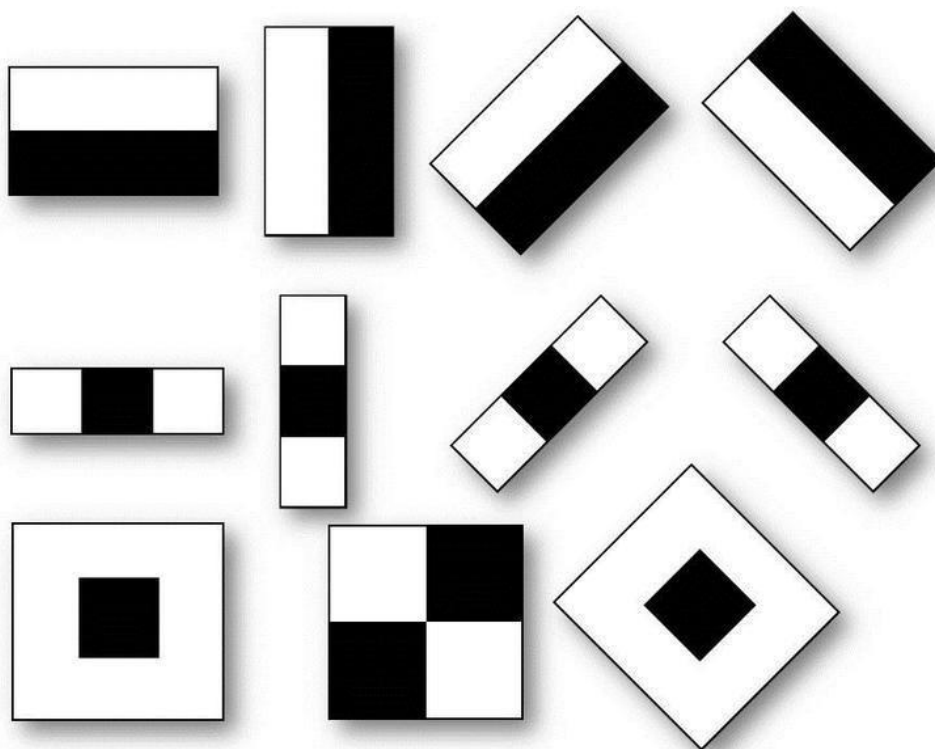


Рисунок 1.3 — Прimitives Хаара

Примитив Хаара представляет собой пару смежных прямоугольных областей. Он располагается на изображении, после чего интенсивности пикселей в каждой области суммируются. Разность между этими двумя суммами является значением соответствующего признака, который определяется его размером и положением на изображении. [1.6]

Также стоит отметить способ определения лица с помощью опорных точек. Первая для детекции опорная точка — моргающие глаза. Она наиболее проста для поиска, так как два глаза расположены симметрично и при этом моргают, что является признаком того, что в этой области есть лицо. Далее программное обеспечение может определить область видеоизображения, которое на самом деле является лицом, используя одну из нескольких моделей лиц. В программах используется несколько моделей лиц с различными параметрами. На самом деле существует множество различных форм лиц, которые примерно подразделяются на овал, прямоугольник, круг, квадрат, треугольник и т.п. Так же существуют другие точки опоры, которые указывают системе, что изображение может содержать лицо. К таким относятся брови,

ноздри, лоб и рот. Пример размеченных опорных точек на лице представлен на Рисунке 1.4.

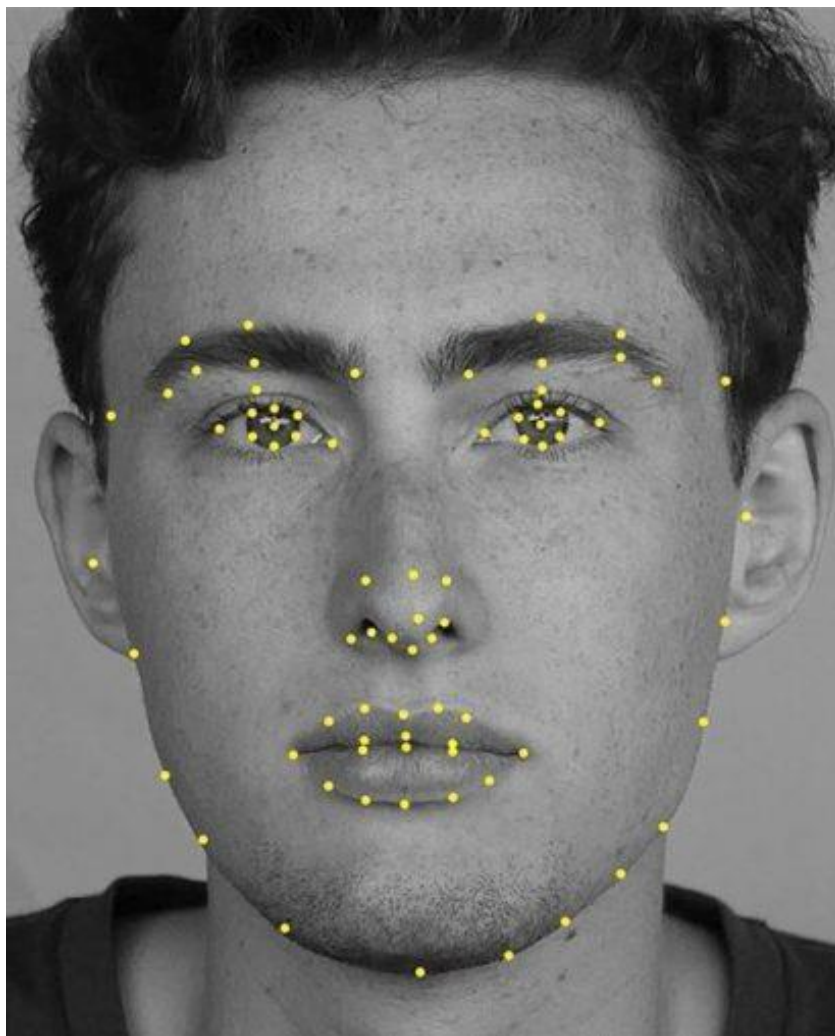


Рисунок 1.4 — Некоторое количество опорных точек, размеченных на лице

После этапа обработки портретного изображения следуют все этапы, непосредственно связанные с анализом — выполнением задач, поставленных при инициировании исследования. Они могут включать в себя:

- кластеризацию;
- классификацию;
- прогнозирование;
- выделение признаков;
- детекцию.

Одной из возможных задач отмечена классификация лицевых изображений, что является предметом представленной работы и ключом к

решению поставленной проблемы. Рассмотрим подробнее классификацию, как один из методов анализа.

1.1.2 Характеристика предмета исследования

Классификация — это область машинного обучения, которая специализируется на решении задач, сформулированных подобным образом: «Существует множество объектов, которые тем или иным образом разбиваются на группы (классы). Дан конечный набор объектов, называемый обучающим набором, и известно, к какому классу каждый из данных объектов принадлежит. Классовая принадлежность остальных объектов неизвестна. Требуется построить алгоритм, который сможет классифицировать любой объект из исходного набора, называемого тестовым». [1.7]

Стоит отметить, что классифицировать объект означает указать класс, к которому данный объект принадлежит. В то же время, классификация объекта — номер или имя класса, созданного алгоритмом классификации в результате применения алгоритма классификации к этому конкретному объекту.

Выделим некоторые алгоритмы классификации, признанные самыми эффективными [1.8]. К ним относятся:

1. К-ближайших соседей, также кратко называемый KNN. В данном алгоритме используются тренировочные наборы данных для поиска k ближайших точек в определённом наборе.
2. Дерево решений. Представляет собой блок-схему, где каждые точки данных по признакам разбиваются на два подмножества, которые таким же образом разбиваются на два подмножества и так далее. Это продолжается до попадания точки в один из классов. В данном пункте следует упомянуть и случайный лес, который является улучшенной версией обозначенного алгоритма.

3. Наивный Байес. С помощью теоремы условной вероятности алгоритм вычисляет шанс, с которым объект попадёт в определённую категорию.
4. Метод опорных векторов. Данные в алгоритме классифицируются согласно их степени полярности. Это позволяет выходить за пределы прогнозирования в координатах x и y .

Представленные алгоритмы могут быть реализованы с помощью различных инструментов. Для решения задач в рамках работы с классификацией изображений, в том числе, с изображениями лиц, чаще всего используют нейронные сети [1.9]. Углубимся в этот вопрос, дав определение данному инструменту.

Искусственные нейронные сети — это математическая модель функционирования традиционных для живых организмов нейросетей, которые представляют собой сети нервных клеток. Подобно биологическому аналогу, в искусственных сетях нейроны выступают основным элементом. Они соединены между собой, образуя слои, число которых может быть разным в зависимости от сложности нейросети и решаемых ею задач.

На изображении ниже (Рисунок 1.5) представлена схема многослойной нейронной сети.

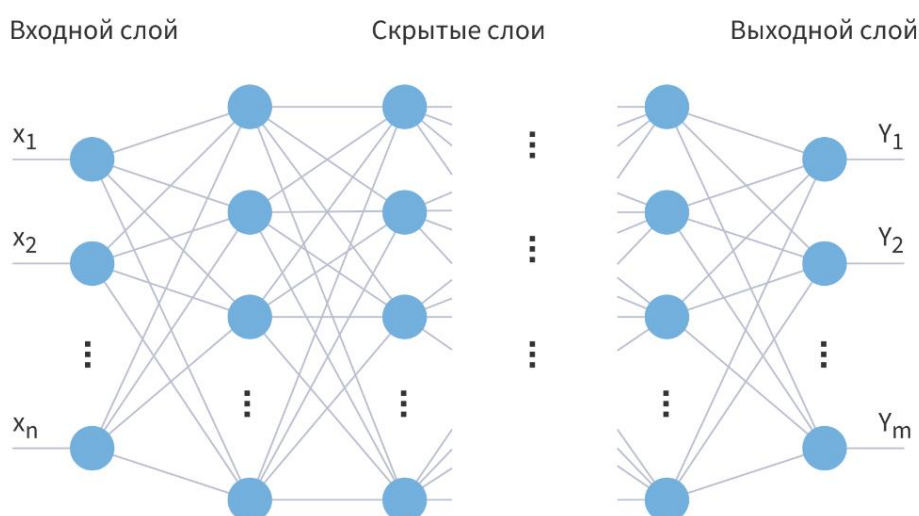


Рисунок 1.5 — Схема многослойной нейронной сети

Заметим, что слои нейронов делятся на один входной, несколько скрытых и один выходной слой [1.10]. Подобное строение также называют «чёрным ящиком», так как нет возможности проанализировать внутреннюю структуру скрытого слоя, выявив на каждом этапе логику преобразований. Входные данные, поступая в модель нейронной сети, проходят некую трансформацию, превращающую их в выходные данные. Изучим данный механизм подробнее.

Для этого в рамках работы с нейросетями введём такие понятия как вес нейрона и обучение. Итак, вес нейрона — это количество связей одного нейрона с другими. Вес является положительным, если данный узел возбуждает другой, или отрицательным, если, наоборот, происходит подавление узла. Соответственно, узлы с большим весом оказывают более сильное влияние на другие.

Обучение — процесс настройки весов. В ходе обучения происходит определение того, какой вклад внесёт каждый нейрон в выходные данные. Существует два типа обучения нейросетей: контролируемое и неконтролируемое обучение. Также их называют обучением с учителем и без, соответственно. [1.11]

Контролируемое обучение — это процесс обучения нейронной сети с помощью наборов данных, предварительно и достоверно разбитых на классы. Далее рассмотрим механизм работы нейросети именно с данным видом обучения, так как оно позволяет повышать точность классификации за счёт механизма минимизации выходной ошибки.

Выходная ошибка рассчитывается с помощью функции потерь. Функция потерь — это функция, с помощью которой определяют, насколько сильно предсказанный моделью результат отличается от фактического. Среди наиболее распространённых функций потерь можно выделить:

- среднеквадратическую ошибку;
- среднюю абсолютную ошибку;
- кросс-энтропийные потери.

В ходе обучения модели также важным атрибутом является оптимизатор. Им называют алгоритм по нахождению метода достижения лучших результатов обучения и уменьшения значения функции потерь.

Одними из самых популярных оптимизаторов являются:

- стохастический градиентный спуск (SGD);
- оптимизатор импульса (Momentum);
- среднеквадратичное распространение (RMSProp);
- адаптивная оценка момента (Adam).

Данные для контролируемого обучения представляют собой набор пар, каждая пара представляет собой входные и соответствующие им выходные данные. Входные данные могут быть представлены в любом формате, тогда как выходные данные должны представлять собой действительные числа или значения из конкретного неизменяющегося списка данных в строковом формате. В некоторых случаях, допускается представление выходных данных в виде векторов или последовательностей. [1.12]

В первую очередь, в рамках анализа, предстоит решить, как преобразовать объект реального мира в вектор признаков, который сможет распознать машина. Для этого справедливо использование тензоров. Тензором называют математический объект линейной алгебры, заданный в векторном пространстве конечной размерности. Тензоры представляют собой не просто контейнеры для хранения числовых данных. Помимо самих данных, они также содержат информацию о допустимых линейных преобразованиях, которые можно применять к этим данным относительно других тензоров. Примерами таких преобразований или отношений являются векторное и скалярное произведение.

Разбив лицевые изображения на машиночитаемые признаки, рассмотрим, по каким параметрам возможно произвести классификацию лицевых изображений.

Классификация изображений лиц может происходить по разнообразным параметрам. Стоит заметить, что параметры выбираются в соответствии с целями исследования [1.13]. Приведём несколько примеров целей и

сопутствующих параметров, которые можно выделить на лицевых изображениях:

- классификация по возрасту людей — оцениваются возрастные изменения лиц;
- классификация по полу людей — оцениваются вторичные половые признаки;
- классификация по расе людей — оцениваются расовые признаки;
- классификация по хобби людей — оцениваются признаки различных субкультур.

Таким образом, заметим, что классификация изображений с лицами может производиться с целями выделения определённых групп людей. Рассмотрим существующие системы, решающие некоторые проблемы с помощью классификации лицевых изображений.

1.2 Обзор существующих решений

В относительно недавней статье группа исследователей из Google Inc. предложила метод извлечения характеристик лица, названный FaceNet [1.14]. В данном методе каждому изображению сопоставляется 128-мерный вектор из евклидова пространства. Таким образом, евклидово расстояние между двумя векторами является мерой схожести соответствующих им лиц.

Чтобы добиться такого результата, была использована глубокая нейронная сеть, обученная с использованием функции потерь на основе троек (triplet-based loss function). Эта функция принимает в качестве аргументов представления трех изображений: «якорное» изображение некоторого человека ($f(x_i^a)$), еще одну фотографию с тем же лицом ($f(x_i^p)$) и изображение другого человека ($f(x_i^n)$). Чтобы получить нужный результат, необходимо в процессе обучения добиться выполнения определённого условия.

Так как выполнение условия для всех существующих троек требует слишком сложного вычислительного процесса, в набор данных для обучения входят только вручную отобранные тройки, для которых это неравенство нарушается. Данный метод крайне удобен тем, что позволяет использовать классические алгоритмы кластеризации (например, KNN) без дополнительных модификаций.

Рассмотрим ещё одну систему извлечения характеристик лица. При разработке системы исследователи ориентировались на подход, использованный в FaceNet. Была использована глубокая нейронная сеть той же архитектуры, при обучении применялась функция потерь на основе троек.

Система была разработана в 2016 году в университете Карнеги-Меллона и получила название OpenFace [1.15]. Она ориентирована на работу в реальном времени, в том числе на мобильных устройствах, при относительно малом количестве доступных данных (фотографий) для обучения.

В то же время, чтобы компенсировать небольшой размер набора данных для обучения, проводится дополнительное выравнивание изображения лица. С помощью детектора ключевых точек лица библиотеки `dlib1` на изображении обнаруживаются точки, соответствующие углам глаз и носу человека на фотографии. Затем они аффинными преобразованиями выравниваются в соответствии с нормализованным представлением лица [1.16]. Отметим, что такое аффинное преобразование. Преобразование плоскости можно считать аффинным, если оно непрерывно, взаимно однозначно и каждая её прямая отображается в прямую. Взаимно однозначность преобразования означает, что различные точки переходят в различные точки, и каждая точка имеет свой образ. Частным случаем аффинных преобразований считаются движения, которые не изменяют расстояния между точками. Например, это могут быть параллельные переносы, повороты, различные симметрии и их комбинации.

После произведения аффинного преобразования размер изображения изменяется до 96x96 пикселей. Визуализацию данного процесса можно увидеть на Рисунке 1.6.

Входное изображение с опорными точками



Опорные точки



Аффинное преобразование



Рисунок 1.6 — Процесс выравнивания изображения лица, используемый в OpenFace

Приведём систему, описанную Трофименко Я. М., Виноградовой Л. Н. и Ершовым Е. В. в 2019 году и предназначенную для идентификации лиц в видеопотоке [1.17]. Идентификация лиц выделяется как отдельная область решений, однако использует те же инструменты, что и алгоритмы классификации. Технически, идентификация лиц производит классификацию лицевых изображений, где классами является набор характерных признаков уникального человека.

Также может показаться, что задача, решённая для видеопотока, не может быть решена на массиве изображений. Однако отметим, что видеопоток — это непрерывный поток изображений, который можно продифференцировать для выявления отдельных изображений и составления массива из них.

Соответственно, данная система является смежной с предметом нашего исследования.

Рассмотрим подробнее приведённую систему. В качестве решаемых проблем, авторы алгоритма выделили:

1. Проблему освещенности. Результаты идентификации сильно зависят от положения источника света относительно лица и общей освещенности в кадре. При недостаточном освещении почти невозможно выделить лицо на изображении, а при интенсивном могут теряться некоторые черты лица.
2. Проблему цветопередачи изображения. Она возникает вследствие того, что для точной цветопередачи необходимо отрегулировать баланс белого камеры в соответствии с цветовой температурой используемого источника света.
3. Проблему разрешения изображения. Состоит в поиске настолько высокого разрешения, чтобы, при наиболее точной идентификации, не возникало чрезмерного увеличения размеров изображения.
4. Проблему расположения камеры. В области обзора не должно находиться объектов, попадающих между камерой и лицом.
5. Проблему положения лица относительно камеры. Угол поворота головы к камере может серьезно влиять на идентификацию, особенно в том случае, когда база данных обладает ограниченным набором изображений для конкретного лица.

Реализованный алгоритм решает описанные выше проблемы. Он преобразует изображения на базе существующего решения для идентификации лиц и способствует минимизации влияния изменений освещения и положения лица в кадре на точность идентификации. Блок-схема данного алгоритма представлена на Рисунке 1.7.

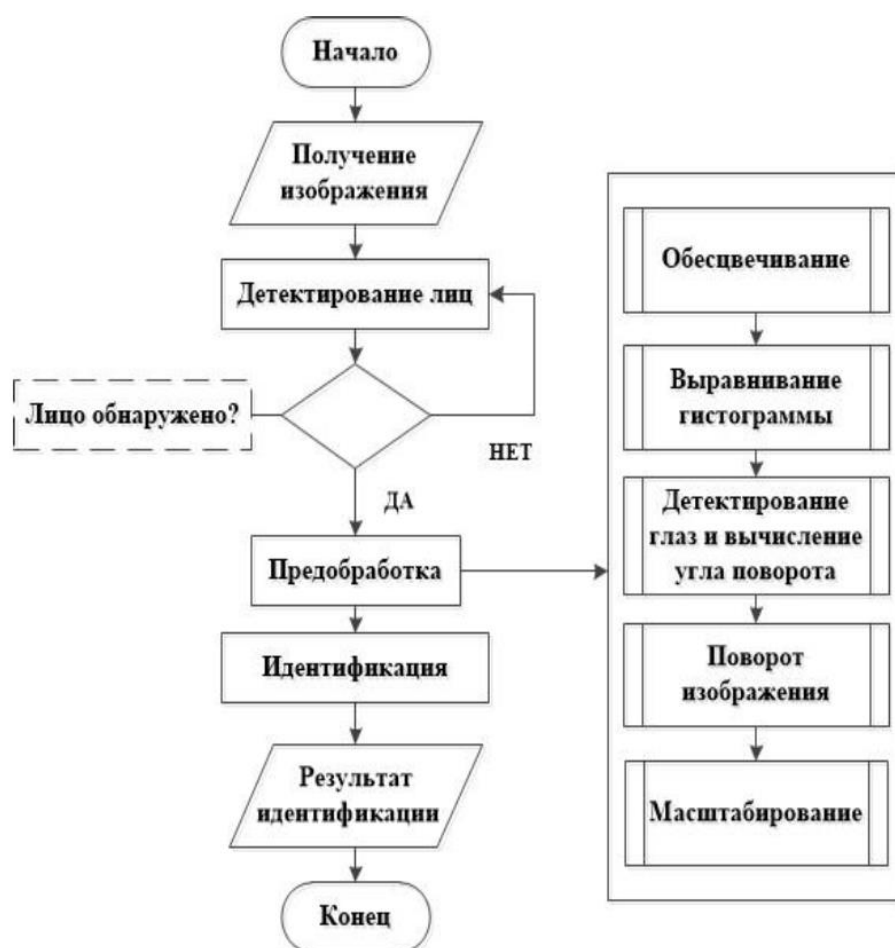


Рисунок 1.7 — Обобщённый алгоритм предобработки и идентификации

Рассмотрим также алгоритм смежной с классификацией области — кластеризацией.

Система кластеризации лицевых изображений в сфере киберкриминалистики разработана Черняковским О.Н. в 2019 году [1.18] и представляет собой конвейер, состоящий из модуля извлечения лиц, модуля создания векторных представлений и модуля кластеризации векторных представлений. Система реализована на языке программирования Python, преследуя цель решения задачи детекции одинаковых людей на различных фотографиях. Пример задачи, для решения которой была разработана система, представлен на Рисунке 1.8.



Рисунок 1.8 — Пример задачи кластеризации изображений лиц

Заметим, что представленные системы решают разнообразные задачи, смежные с классификацией лицевых изображений. Однако они не учитывают возможную проблему, связанную со спецификой лиц, которую мы подробнее затронем во второй части данной работы. Лицевые изображения выдуманных и нарисованных в 2D или 3D стиле персонажей часто могут быть спутаны с изображениями реально существующих людей. Таким образом, перед нами встаёт следующая задача.

1.3 Формулировка исследовательской задачи подхода к разработке нейронной сети

Необходимо написать классификатор, разделяющий входной массив предобработанных лицевых изображений на рисованные и реальные человеческие лица.

2 АНАЛИТИЧЕСКИЙ РАЗДЕЛ

Для написания классификатора следует понимать, что процесс разработки состоит из нескольких этапов: программирование, обучение, анализ результатов работы модели с использованием тестовой выборки — а также, при наличии багов или неудовлетворительного результата работы модели, возможно повторение предыдущих пунктов. Таким образом, для правильного подбора инструментария разработки прежде следует описать входные данные.

2.1 Анализ и характеристика входных данных

Отметим, что для работы модели понадобятся изображения, которые в дальнейшем будут подвергаться обработке, и некоторые булевы значения. Данные булевы значения необходимы для обучения с учителем и будут отражать ожидаемые выходные данные для системы. Таким образом, входные данные представляют собой массив изображений, а также массив булевых значений.

Массив булевых значений отвечает за однозначное определение изображения, относящее его к одному из двух классов. Это необходимо для качественного обучения модели. Стоит отметить, что каждому изображению соответствует значение «0» или «1» (соответственно, их индексы в рамках массивов совпадают). «0» означает, что на изображении представлено рисованное лицо, тогда как «1» означает, что это фотография реального человека. Значения присваиваются в процессе сбора данных, так как изображения собираются последовательно: сначала изображения человеческих лиц, а далее — изображения персонажей. Это исключает возможность ошибки присваивания некорректного значения класса, в отличие от случая, если данные были бы собраны в хаотичном порядке.

Для уточнения процесса сбора обоих массивов изображений, рассмотрим каждый из них в отдельности.

2.1.1 Изображения человеческих лиц

Начнём описание со сбора массива изображений человеческих лиц, так как в открытом доступе представлено большое количество предсобранных массивов [2.1], из которых важно подобрать подходящий для целей данного исследования.

Для выбора подходящего датасета сравним несколько самых крупных массивов изображений человеческих лиц (Таблица 2.1).

Таблица 2.1 — Датасеты человеческих лиц

Название датасета	Год выпуска	Количество изображений	Количество личностей	Лицензия
DigiFace1M	2022	1.2 миллиона	110 тысяч	Доступен для некоммерческих исследований
CelebA	2015	~200 тысяч	~10 тысяч	Доступен для некоммерческих исследований
VGG Face2	2018	~3.3 миллиона	~9 тысяч	Доступен для некоммерческих исследований
UMDFaces	2017	~370 тысяч	~8 тысяч	Доступен для некоммерческих исследований
Wider Face	2018	~32 тысячи	~400 тысяч	Доступен для некоммерческих исследований

Исходя из представленной выше таблицы, нетрудно заметить, что массив лицевых изображений DigiFace1M обладает большим количеством преимуществ по сравнению с остальными датасетами. Однако заметим основную особенность данного датасета, отмеченную в его описании [2.2]. «Набор данных представляет собой коллекцию из более чем миллиона различных синтетически созданных

изображений лиц для распознавания лиц», что значит, что изображения лиц, похожих на человеческие, были сгенерированы.

В рамках исследований, связанных с детекцией лиц на изображениях, качественно проведённая искусственная генерация не повлияет на качество результатов. Однако обучение классификатора для различия человеческих и рисованных лиц требует входного массива, собранного из реальных человеческих лиц. Таким массивом из представленных является Wider Face. [2.3]

Отметим его основные преимущества по сравнению с другими массивами данных:

- большое количество изображений;
- разнообразие представленных личностей;
- открытый доступ для некоммерческих исследований.

Рассмотрим выбранный датасет подробнее. Набор данных Wider Face — это набор эталонных данных для распознавания лиц, изображения которого выбираются из общедоступного набора данных Wider [2.4]. Для составления датасета было выбрано 32203 изображения, в каждом из которых были помечены 393703 лица с высокой степенью изменчивости масштаба, позы, преград, выражений, макияжа и освещения, как показано на примерах изображений (Рисунок 2.1).



Рисунок 2.1 — Примеры изображений, входящих в датасет Wider Face

Разнообразие используемых изображений позволит нам обучить классификатор распознавать человеческое лицо в различных ситуациях, которые

могли бы быть ему недоступны с более узким набором входных данных (например, если бы на изображениях был представлен только один человек или если бы все лица были повёрнуты в анфас).

Но фотографии с человеческими лицами составляют только половину от всех подаваемых на вход изображений, стоит также отметить процесс сбора второй части, а именно, изображений рисованных лиц.

2.1.2 Изображения рисованных лиц

В отличие от процесса сбора данных человеческих лиц, в котором требовалось лишь проанализировать существующие датасеты и выбрать наиболее подходящий, сбор изображений рисованных лиц потребует провести непосредственно.

Обозначим этапы формирования датасета (далее они будут включены в общую архитектуру системы):

1. Поиск видеофрагментов, включающих в себя рисованных персонажей, в открытом доступе.
2. Разбиение видео на кадры (изображения).
3. Детекция и обрезание области лица на каждом изображении.
4. Сбор полученных изображений в датасет.

Рассмотрим каждый из этапов в отдельности, для того чтобы привести необходимый инструментарий.

На первом этапе никакой инструментарий не понадобится — видеоматериалы следует собирать, исходя из собственных соображений, скачивать и сохранять в едином формате в общей созданной для этого папке. На втором этапе предполагается использование языка программирования Python и библиотек для анализа видеофайлов, работы с временными переменными и многомерными массивами (OpenCV, time и numpy, соответственно) [2.5]. Далее, для детекции и обрезания области лица, разумно использовать уже упомянутую

библиотеку OpenCV, а запись полученных файлов осуществлять с помощью модуля os.

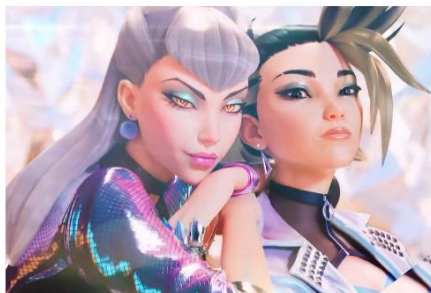
Для точности обучения классификатора видеоматериалы должны включать в себя несколько различных стилей. В мультипликации как наиболее различающиеся стили можно выделить:

- японскую анимацию;
- советскую анимацию;
- итальянскую анимацию
- проекты Disney;
- проекты Riot;
- проекты Barbie. (Рисунок 2.2)

Япония



Riot



Италия



Barbie



СССР



Disney



Рисунок 2.2 — Стили видеофрагментов, собираемых для входных данных

Таким образом, мы определили формат и порядок сбора всех данных, необходимых для поступления на вход модели.

2.2 Описание этапов разработки модели

Сбор и предобработка входных данных составляют первый этап в разработке модели. В исследовательском разделе уже была представлена архитектура типичного конвейера по обработке изображений лиц, далее же следует смоделировать конвейер, актуальный в рамках исследования. На Рисунке 2.3 представлена схема конвейера для решения задач, поставленных в ходе данной работы.

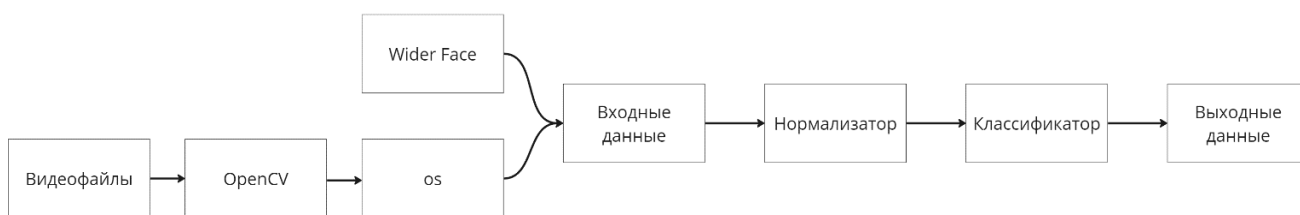


Рисунок 2.3 — Схема архитектуры конвейера

Рассмотрим каждый из его этапов, обозначив необходимый инструментарий и технологии его разработки и обучения.

Этапы до получения массива входных данных были описаны в разделе 2.1. На вход в нормализатор получает уже собранный датасет. Задачей нормализатора в данном конвейере является нормализация изображений, а конкретно, поворот лиц в правильное положение для достижения стандартизации. Для этих целей предполагается использование Face Normalization Model (модель нормализации лиц, далее FNM): решения, написанного китайскими программистами [2.6]. Оно представляет собой открытый исходный код с имеющимися обученными моделями.

После нормализации следует приступить к этапу классификации каждого изображения в массиве на рисованное и реальное. В рамках разработки конвейера потребуется дополнительное обучение классификатора, для чего наш

датасет необходимо разбить на обучающую и тестовую выборку. В качестве классификатора изображений воспользуемся библиотекой Tensorflow. Она представляет собой программное обеспечение с открытым исходным кодом для высокопроизводительных числовых вычислений и отлично подходит для разработки нейронных сетей [2.7]. Отметим, что для её использования необходимо предварительно получить изображения и метки. Изображения должны быть нормализованы, а метки должны быть закодированы в горячем режиме, что учтено в описанных выше этапах реализации системы.

Перечисленные инструментальные средства для использования в ходе решения поставленных задач являются оптимальными, и были выбраны, как лучшее из существующих решений. Рассмотрим в отдельном порядке каждую из упомянутых систем, выделив её особенности преимущества.

2.3 Обоснование выбора инструментальных средств разработки

Прежде всего отметим основной язык программирования, выбранный для разработки модели, так как он является фундаментальным элементом для остальных систем.

2.3.1 Обоснование выбора языка программирования Python

Известно, что Python — востребованный язык программирования, широко применяемый в интернет-приложениях, программной инженерии, анализе данных и машинном обучении [2.8]. Разработчики предпочитают использовать Python из-за его преимуществ перед другими языками, программы на Python доступны для бесплатного скачивания, они совместимы со всеми видами операционных систем и сокращают время разработки.

Основными положительными чертами данного языка разработки являются:

- лёгкость освоения;
- простота визуального восприятия;
- кроссплатформенность;
- скорость разработки;
- универсальность;
- разнообразие инструментов (библиотек);
- масштабируемость.

Однако он имеет и свои недостатки, такие как:

- медленная работа;
- трудность переноса проектов на другие системы;
- ресурсоёмкость;
- необходимость использования резервного языка программирования.

Все недостатки Python нивелирует за счёт использования дополнительных библиотек, код внутри которых написан на быстрых низкоуровневых языках. Таким образом, комбинируется удобство и читаемость языка Python, и скорость работы таких языков, как C или C++, что делает Python одним из самых лучших языков для разработки систем различного назначения. Библиотеки обеспечивают поддержку многих функций, в том числе, компьютерного зрения. Одной из библиотек на Python, работающих с изображениями, является OpenCV.

2.3.2 Обоснование выбора библиотеки OpenCV

Для обоснования выбора библиотеки OpenCV в качестве библиотеки, предоставляющей возможности работы с изображениями, стоит обратить внимание, что существует разница между классификацией изображений и обнаружением объектов. Если классификация изображений — это отнесение

изображения к какой-либо категории (классу), то обнаружение объектов — это обнаружение, классификация и локализация объектов в реальных изображениях. В рамках обнаружения объектов одним из основных алгоритмов является YOLO, однако собранные изображения для обработки предполагают нахождения лишь одной природы — лиц [2.9]. Соответственно, алгоритм YOLO оказывается избыточным и нам следует обратиться непосредственно к библиотеке OpenCV.

Аналогов данного инструментария практически не существует [2.10], однако, в любом случае, следует выделить основные особенности для понимания причин актуальности библиотеки:

- наличие более 2500 оптимизированных алгоритмов компьютерного зрения и машинного обучения;
- возможности обнаружения и распознавания лиц, идентификации объектов, классификации действий человека в видео, отслеживания движений камеры, отслеживания движущихся объектов, извлечения трехмерных моделей объектов, создания трехмерных облаков точек из стереокамер и т.д;
- широкое использование в компаниях, исследовательских группах и государственных органах (в том числе, Google, Yahoo, Microsoft, Intel);
- наличие интерфейсов C ++, Python, Java и MatLab, поддержка Windows, Linux, Android и Mac OS;
- ориентация на приложения для визуализации в реальном времени и использует инструкции MMX и SSE, когда они доступны.

Благодаря этим особенностям, данную библиотеку можно считать эффективным средством для сбора массива изображений [2.11]. После этого, для увеличения точности обучения классификатора, необходимо нормализовать полученные изображения. Как было отмечено, для этого хорошо подойдет решение FNM.

2.3.3 Обоснование выбора сервиса FNM для нормализации изображений

CLIP является современным решением в проблеме классификации изображений. Исследователи из OpenAI продемонстрировали, что обучение на парах «изображение-текст», даже с использованием необработанного текста, является многообещающей альтернативой традиционным методам классификации. Задача предварительного обучения заключается в том, чтобы для каждого изображения из пары угадать, какое текстовое описание соответствует данному изображению. Нейронная сеть, способная решать эту задачу, была предобучена на 400 миллионах пар «изображение-текст», собранных из интернета [2.12]. Такая сеть сможет ранжировать визуальные и текстовые концепции по метрике сходства. После этого предобученную модель можно легко использовать для широкого спектра задач в режиме «обучения без обучения». Принцип работы модели CLIP можно наблюдать на Рисунке 2.4.

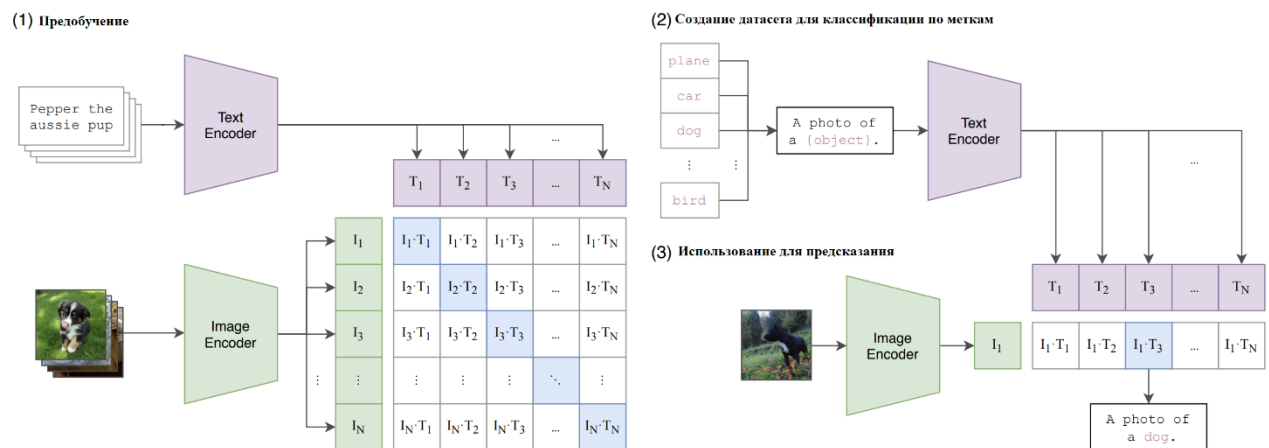


Рисунок 2.4 — Принцип работы модели CLIP

После обзора инструментария данного этапа обратимся к библиотеке для написания классификатора на основе нейронной сети.

2.3.4 Обоснование выбора библиотеки Tensorflow

Изначально разработанная исследователями и инженерами команды Google Brain в рамках искусственного интеллекта Google, библиотека Tensorflow обладает стабильной базой для машинного обучения и глубокого обучения [2.13], а гибкое числовое ядро используется во многих других научных областях.

Отметим основные преимущества данной библиотеки перед аналогами:

- наличие большого количества руководств и документации;
- мощные средства мониторинга процесса обучения моделей и визуализации (Tensorboard);
- поддержка большим сообществом разработчиков и техническими компаниями;
- обеспечение обслуживания моделей;
- поддержка распределенного обучения;
- обеспечение вывода на устройства с низкой задержкой для мобильных устройств (TensorFlow Lite).

Благодаря этим особенностям, Tensorflow хорошо подходит для сложных проектов, таких как создание многослойных нейронных сетей, и часто применяется в различных задачах, связанных с машинным обучением. Например, она используется для распознавания голоса или изображений в таких приложениях для работы с текстом, как Google Translate. Работа с графическими материалами является особой спецификацией работы данной библиотеки, соответственно, она будет актуальна в рамках рассматриваемых задач.

Таким образом, были рассмотрены инструментальные средства, необходимые для реализации создаваемой системы.

3 ТЕХНОЛОГИЧЕСКИЙ РАЗДЕЛ

В предыдущих разделах была представлена исследовательская и аналитическая часть работы, охватывающие как основные понятия, так и технологии, необходимые для реализации практической части. Обозначим порядок действий, которые были осуществлены для выполнения задач технологического раздела.

3.1 Описание реализации разработанной нейронной сети

В первую очередь, стояла задача нахождения входных данных, которые включают в себя датасет человеческих лиц и видеофрагменты с рисованными лицами.

Далее потребовалось обработать полученные данные. Для датасета человеческих лиц — вырезать лица из изображений, используя алгоритм MTCNN [3.1]. Для видеофрагментов — предварительно разбить видео на изображения, а после из изображений с помощью того же алгоритма, вырезать лица. Полученные лица были разделены на обучающую и тестовую выборку и сохранены с добавлением метки «human» для изображений лиц людей и метки «character» для изображений лиц персонажей, соответственно.

С собранным датасетом человеческих и рисованных лиц предстояло классифицировать с помощью предобученного алгоритма CLIP [3.2], после чего проанализировать и оценить точность предсказания, а далее — обучить модель. После обучения был повторён анализ точности предсказания и сделаны выводы насчёт достижения поставленных задач.

Теперь опишем каждый из этапов более подробно, приводя результаты и фокусируясь на особенностях разработки.

3.1.1 Сбор и предобработка входных данных

Как было отмечено выше, сбор данных включал в себя формирование части датасета с лицами людей и с лицами персонажей.

Widerface — датасет, включающий в себя изображения людей в различных ситуациях, — представлен в открытом доступе на сайте разработчиков [3.3]. Внешний вид официального сайта разработчиков данного датасета представлен на Рисунке 3.1.

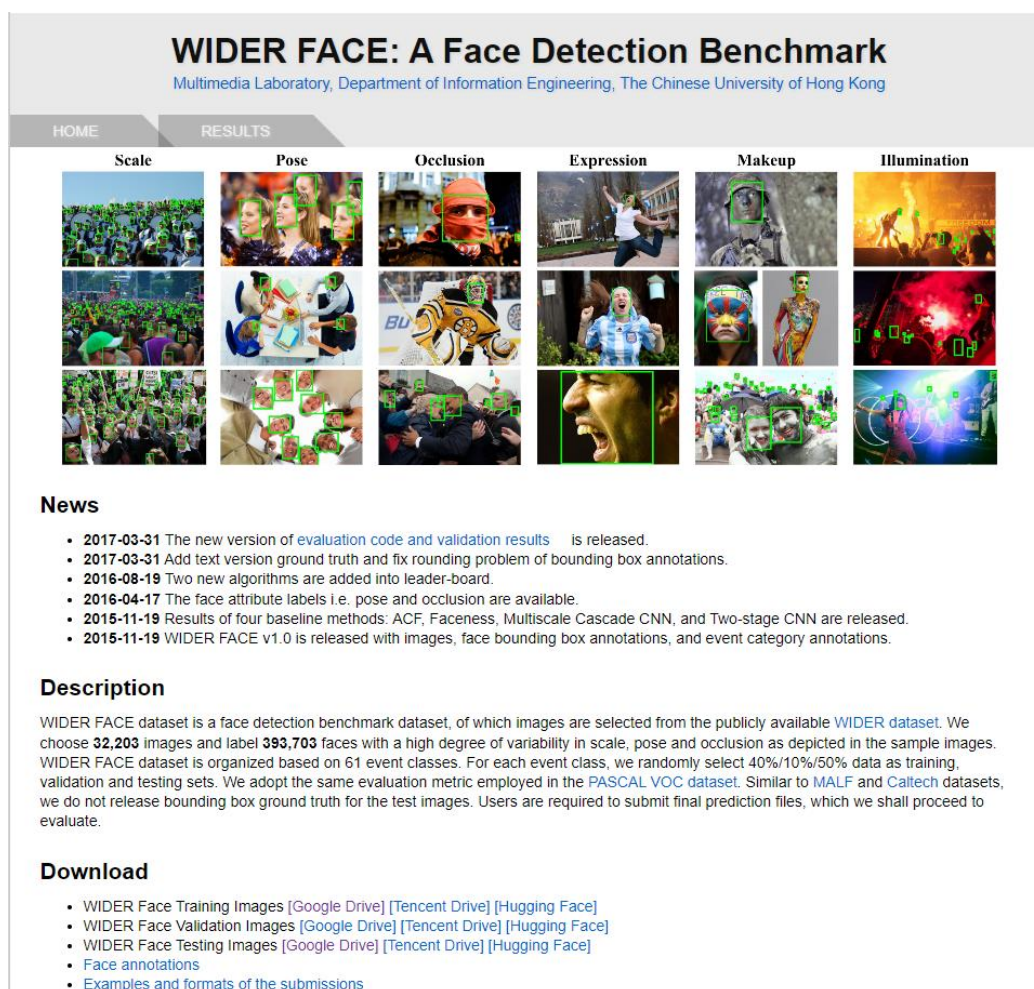


Рисунок 3.1 — Внешний вид официального сайта датасета Widerface

Там же расположены ссылки на скачивание тестовой, валидационной и тренировочной выборок. Для получения тестовой выборки было принято решение брать только часть датасета, из-за ограничений устройства по памяти и производительности [3.4]. Таким образом, была взята тестовая выборка. Внутри

полученная тестовая выборка делится на папки с изображениями, разбитыми по категориям (Рисунок 3.2).

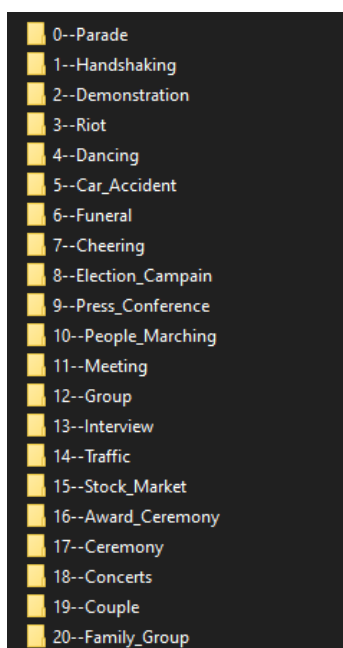


Рисунок 3.2 — Пример категорий изображений, входящих в датасет Widerface

Среди категорий можно отметить такие как «парад», «рукопожатие», «демонстрация», «танцы», «похороны», «концерты» и другие. Это подтверждает факт, описанный в аналитической части: представленные изображения располагают широким спектром ситуаций, включающих в себя людей. [3.5]

Далее рассмотрим процесс сбора видеофрагментов из мультипликации. Фрагменты были собраны из различных открытых источников и сохранены в формате .avi, .mp4 или .mkv в отдельные папки. Длительность каждого из фрагментов составляет приблизительно 60 минут.

После сбора следует этап необходимой обработки данных.

3.1.2 Нормализация изображений

В первую очередь, опишем обработку видеофрагментов, так как, после получения изображений из них, они были обработаны так же, как и изображения из датасета Widerface. [3.6]

Для извлечения изображений из видеофрагментов были использованы такие библиотеки, как `time`, `os` и `opencv`. Код представлен в Приложении Б, а на Рисунке 3.3 можно наблюдать результат выполнения кода на примере фрагментов из мультипликации СССР.

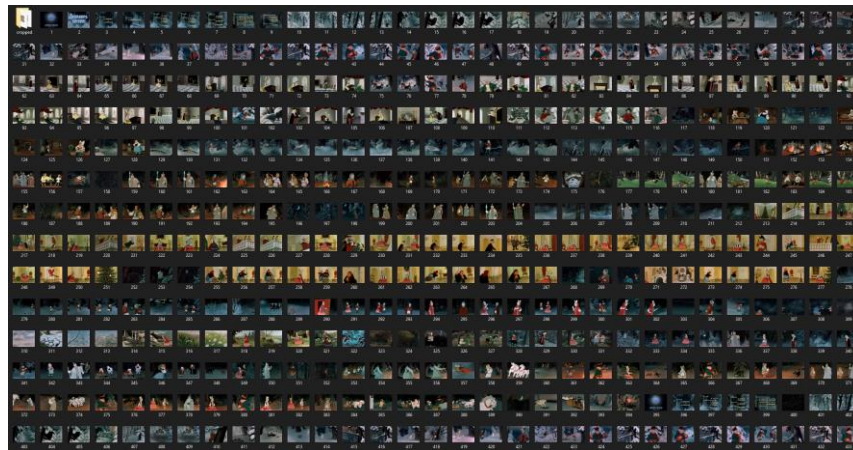


Рисунок 3.3 — Результат вырезки изображений на примере мультипликации СССР

После получения изображений вырезаны кадры, включающие лица.

Для этого были использованы несколько библиотек, в том числе `facenet` `pytorch`, включающая в себя модель, обученную на датасете `vggface` [3.7]. В ходе выполнения программы модель выделяет область размером 96x96 пикселей, в которой находится лицо и сохраняет полученное изображение в формате `.jpg`. Стоит отметить, что при проведении детекции изображение переводится в тензор и нормируется, из-за чего при переводе обратно в изображение искажаются цвета и формы. Искажение сильно влияет на читаемость изображений. Чтобы этого не происходило, в код добавлена функция `_inverse_norm`, которая проводит операцию, обратную нормированию тензора. [3.8]

Код представлен в Приложении В, а также пример вырезанных лиц можно увидеть на Рисунках 3.4 и 3.5.

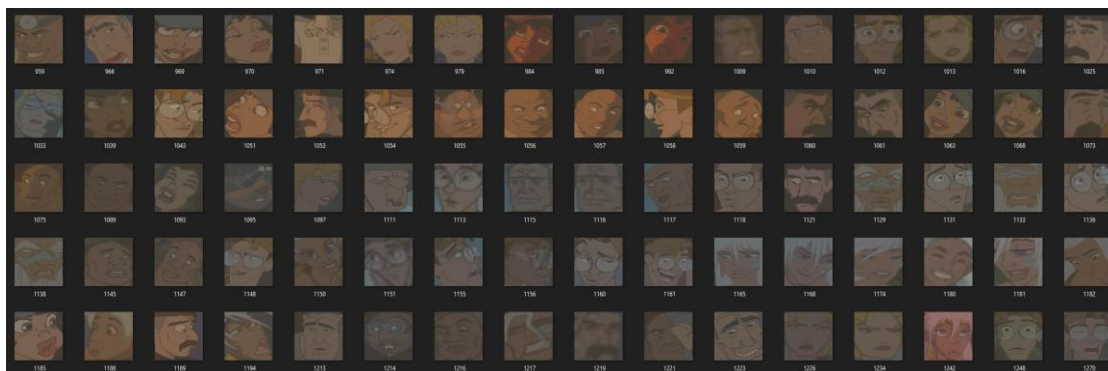


Рисунок 3.4 — Результат детекции лиц на изображениях на примере мультипликации Disney



Рисунок 3.5 — Результат детекции лиц на изображениях на примере Widerface

После вырезки изображения были подвергнуты нормализации с помощью алгоритма FNM. Далее полученные изображения были организованы в структурированный датасет. Датасет состоит из отдельных каталогов для тренировочных и тестовых данных, а внутри каждого из них находятся подкаталоги для каждой категории изображений. Каталоги названы в соответствии с категорией изображений (“human” и “character”, соответственно).

Отметим, что, так как изображений в Widerface в семь раз больше, чем собранных изображений с персонажами, следует добавить код для сохранения каждого седьмого лица, вырезанного из датасета человеческих изображений. Это позволяет охватить весь спектр тематик и соблюсти баланс между рисованными и реальными лицами. [3.9]

Характеристики собранного датасета проверяются с помощью функции `load_dataset`. Результат вывода данной функции можно наблюдать на Рисунке 3.6.

```
DatasetDict({
  train: Dataset({
    features: ['image', 'label'],
    num_rows: 3817
  })
  test: Dataset({
    features: ['image', 'label'],
    num_rows: 1908
  })
})
```

Рисунок 3.6 — Информация о собранном датасете

На основе полученного датасета происходит обучение классификатора изображений на рисованные и реальные лица.

3.1.3 Обучение классификатора

Для обучения используется предобученная модель CLIP [3.10]. В первую очередь, происходит оценка начальной точности модели. Для этого модель была запущена на тестовой выборке и оценена точность предсказания. Стоит отметить, что для корректной работы модели значения классов были переведены в формат “a photo of a {class name}”, а также дополнительно изображения и значения классов были токенизированы.

Модель определяет вероятность вхождения изображения в каждый из классов “human” и “character”. Для определения точности было найдено наибольшее из значений вероятности и записано значение класса в отдельный список. После значения в изначальном списке и предсказанном были сравнены, а каждое совпадение значений записано как «1» и несовпадение как «0». После делится сумму всех значений на их количество, тем самым находится точность предсказания.

В результате выполнения предсказания, модель показала точность в ~70%. Код представлен в Приложении Г.

3.1.3 Классификация изображений

С целью увеличения точности модели, произведено её обучение, так называемый «тюнинг» параметров. Код обучения представлен в Приложении Д. Отметим, что в качестве оптимизатора был выбран Адам [3.11]. Обучение проводилось в 30 эпох, в каждую из которых при первом проходе подсчитывалось значение функции потерь, при обратном — высчитывалось изменение весов для уменьшения значения функции потерь.

Для экономии памяти сохранялись параметры каждой третьей эпохи модели. Процесс обучения и сохранённые модели представлены на Рисунках 3.7 и 3.8, соответственно.

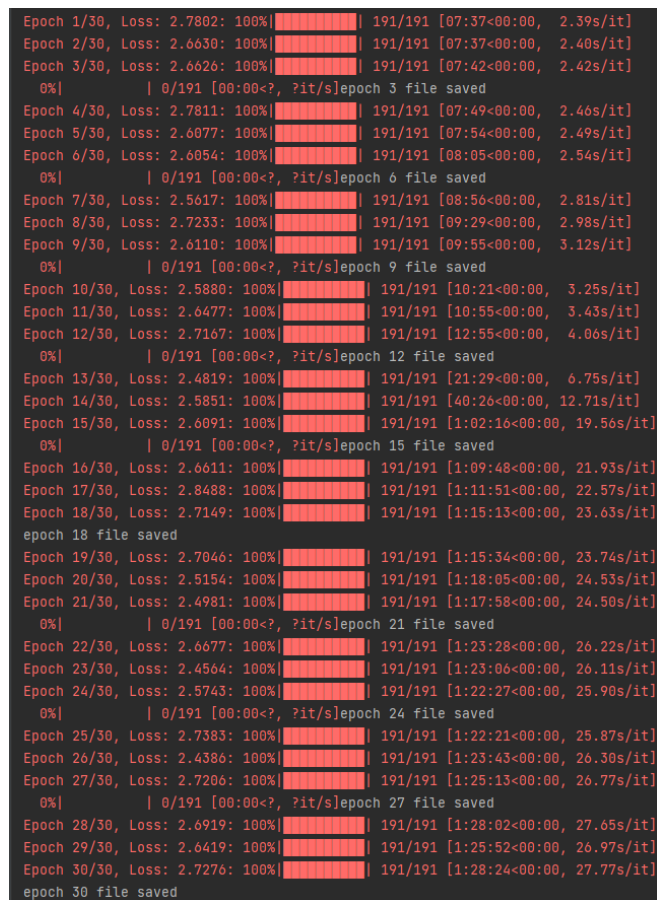


Рисунок 3.7 — Процесс тюнинга модели

 model_3_2.662647008895874	10.05.2024 7:42	Файл ".pt"	1 773 127 КБ
 model_6_2.6054468154907227	10.05.2024 8:06	Файл ".pt"	1 773 127 КБ
 model_9_2.6109933853149414	10.05.2024 8:34	Файл ".pt"	1 773 127 КБ
 model_12_2.7166929244995117	10.05.2024 9:09	Файл ".pt"	1 773 127 КБ
 model_15_2.6090636253356934	10.05.2024 11:13	Файл ".pt"	1 773 127 КБ
 model_18_2.7148680686950684	10.05.2024 14:50	Файл ".pt"	1 773 127 КБ
 model_21_2.4981443881988525	10.05.2024 18:42	Файл ".pt"	1 773 127 КБ
 model_24_2.5742907524108887	10.05.2024 22:51	Файл ".pt"	1 773 127 КБ
 model_27_2.7206311225891113	11.05.2024 3:02	Файл ".pt"	1 773 127 КБ
 model_30_2.727597236633301	11.05.2024 7:25	Файл ".pt"	1 773 127 КБ

Рисунок 3.8 — Сохранённые параметры модели на разных эпохах

Таким образом, было проведено обучение модели. Далее был выбран один из сохранённых наборов параметров.

3.2 Анализ полученных результатов

Использованы параметры модели с последней эпохи для прогнозирования на тестовой выборке, так как имеет наименьшее значение функции потерь. Код для прогнозирования аналогичен коду, представленному в Приложении Д, за исключением того, что потребовалось добавить чтение параметров модели из файла .pt с помощью функции `torch.load()`. С помощью загруженных параметров, на тестовой части выборки модель предсказала вхождение каждого из изображений в класс изображений людей или изображений персонажей, соответственно.

После этого точность предсказания была оценена способом, описанным выше. Для обученной модели она составила $\sim 80\%$. Отметим увеличение точности предсказания модели после обучения на собранных данных на $\sim 10\%$. [3.12]

Также составим распределение уверенности модели в предсказании и сравним с распределением уверенности модели до тюнинга (Рисунок 3.9).

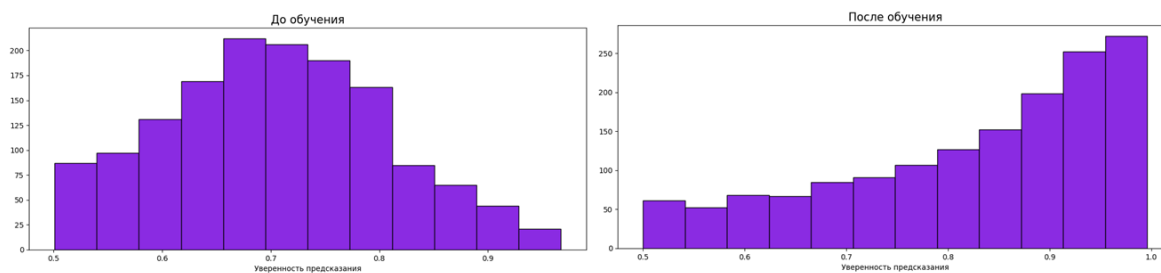


Рисунок 3.9 — Распределение уверенности модели до и после обучения

Отметим значительное увеличение уверенности предсказания после обучения. До обучения пик имеет значение близкое к 65 %, тогда как после обучения стремится к 100 %.

Также стоит обратить внимание на смещение максимального значения уверенности. Уверенность — это распределение значений вероятности попадания в класс, с которыми изображения были определены в верный класс. До обучения максимальное значение уверенности составляло ~95 %, тогда как после дошло до ~99 %.

Посчитаем количество истинно положительных, истинно отрицательных, ложно положительных и ложно отрицательных значений обученной модели и, на их основе, составим метрики precision, recall и F-1 меру. Полученные значения метрик представлены на Рисунке 3.10.

```
precision = 0.7313278008298755
recall = 0.8576642335766423
F-1 = 0.7894736842105263
```

Рисунок 3.10 — Значения метрик precision, recall и F-1 меры

Отметим, что значение каждой из метрик превышает 0.73, что можно считать эффективным показателем качества работы модели.

Таким образом, можно сделать вывод, что обученный классификатор может эффективно определять принадлежность изображения к классу человеческих или рисованных лиц.

ЗАКЛЮЧЕНИЕ

В результате данной выпускной квалификационной работы была рассмотрена область анализа изображений лиц, а также проведён обзор существующих решений в обозначенной области. В процессе анализа была определена методика сбора датасета и обоснован выбор инструментальных средств разработки. Далее была разработана нейронная сеть классификации для различия реальных и рисованных лиц.

Цель выпускной квалификационной работы — разработка метода классификации, различающего рисованные и реальные изображения лиц.

В ходе выполнения данной выпускной квалификационной работы выполнены следующие задачи:

1. Охарактеризована предметная область.
2. Описаны существующие технологии для работы с лицевыми изображениями.
3. Проведён обзор существующих решений.
4. Выбраны сопутствующие средства для разработки.
5. Обоснован выбор средств разработки.
6. Описана реализация программного кода.
7. Подведены итоги разработки.

Разработанный метод открывает новые возможности для дальнейшего совершенствования систем компьютерного зрения и повышения их точности. Он позволяет более эффективно обрабатывать и анализировать визуальную информацию, получаемую от различных источников, таких как камеры, сенсоры и другие устройства. Применение данного метода может привести к значительному улучшению качества и надежности систем компьютерного зрения, используемых в таких областях, как робототехника, беспилотные транспортные средства, медицинская диагностика и многих других.

As a result of this graduation qualification work, the field of facial image analysis was examined, and a review of existing solutions in the specified area was conducted. During the analysis, the methodology for collecting the dataset was determined, and the choice of development tools was justified. Further, a neural network classification was developed to distinguish between real and drawn faces.

The purpose of the graduation qualification work is to develop a classification method that can distinguish drawn and real face images.

During the implementation of this graduation qualification work, the following tasks were completed:

1. The subject area was characterized.
2. Existing technologies for working with facial images were described.
3. A review of existing solutions was conducted.
4. The accompanying tools for development were selected.
5. The choice of development tools was justified.
6. The implementation of the program code was described.

The results of the development were summarized.

The developed method opens up new opportunities for further improving computer vision systems and increasing their accuracy. The successful implementation of this facial analysis model demonstrates the potential for continued advancements in the field of computer vision through the application of sophisticated machine learning approaches. Further research and refinement of this technique could lead to even more robust and versatile facial analysis capabilities in the future. By leveraging sophisticated facial analysis techniques, this classification model can contribute to enhancing the accuracy and robustness of a wide range of applications, such as biometric authentication, facial recognition, and digital forensics. Furthermore, the successful implementation of this method demonstrates the potential for continued innovation and progress in the field of artificial intelligence, as researchers strive to push the boundaries of what is possible in automated visual perception and understanding.

СПИСОК ИСТОЧНИКОВ

1 ИССЛЕДОВАТЕЛЬСКИЙ

1.1 Shaip. Настройка конвейера данных для надежной и масштабируемой модели машинного обучения [Электронный ресурс]. — Режим доступа: <https://ru.shaip.com/blog/setting-up-data-pipeline-to-build-scalable-ml-model/>

1.2 Хабр. Современное распознавание лиц с глубинным обучением [Электронный ресурс]. — Режим доступа: <https://habr.com/ru/articles/306568/>

1.3 Код. Находим лица в картинке с веб-камеры [Электронный ресурс]. — Режим доступа: <https://thecode.media/cv2/>

1.4 Microsoft Learn. Распознавание лиц и анализ тональности [Электронный ресурс]. — Режим доступа: <https://learn.microsoft.com/ru-ru/azure/architecture/example-scenario/ai/nifi-sentiment-analysis-face-recognition>

1.5 Центр2М. Распознавание образов и изображений с помощью ИИ [Электронный ресурс]. — Режим доступа: <https://center2m.ru/ai-recognition>

1.6 Цифровой океан. Как работает система распознавания лиц в московском метро [Электронный ресурс]. — Режим доступа: <https://digitalocean.ru/n/licevoj-schet>

1.7 Распознавание. Классификация [Электронный ресурс]. — Режим доступа: www.machinelearning.ru/wiki/index.php?title=Классификация

1.8 Medium. 5 типов алгоритмов машинного обучения [Электронный ресурс]. — Режим доступа: <https://medium.com/nuances-of-programming/5-типов-алгоритмов-машинного-обучение-которые-вы-должны-знать-550343744c92>

1.9 Хабр. Используем MLOps-конвейер [Электронный ресурс]. — Режим доступа: <https://habr.com/ru/companies/vk/articles/780050/>

1.10 Michael Nielsen. Using neural nets to recognize handwritten digits [Электронный ресурс]. — Режим доступа: <http://neuralnetworksanddeeplearning.com/chap1.html>

1.11 ВикиЛогин. Обучение с учителем (Supervised learning) [Электронный ресурс]. — Режим доступа: <https://wiki.loginom.ru/articles/supervised-learning.html>

1.12 Бурков А. Машинное обучение без лишних слов. — Санкт-Петербург: Питер, 2020. — 192 с.

1.13 Как работает распознавание лиц [Электронный ресурс]. — Режим доступа: <https://habr.com/ru/companies/droider/articles/568764/>

1.14 Schroff F. FaceNet: A unified embedding for face recognition and clustering / Schroff F., Kalenichenko D., Philbin J. — IEEE Conference on Computer Vision and Pattern Recognition, 2015 — с. 815–823.

1.15 Amos B. OpenFace: A general-purpose face recognition library with mobile applications / Amos B., Ludwiczuk B., Satyanarayanan M. — 2016 — 18 с.

1.16 Unite.ai. DeepFace для расширенного распознавания лиц [Электронный ресурс]. — Режим доступа: <https://www.unite.ai/ru/deepface-для-расширенного-распознавания-лиц>

1.17 Трофименко Я.М. Алгоритмы предобработки изображений в системе идентификации лиц в видеопотоке / Трофименко Я.М., Виноградова Л.Н., Ершов Е.В. — Череповец: Вестник Череповецкого государственного университета, 2019. — 29 с.

1.18 Чернявский О.Н. Анализ решений задачи кластеризации изображений лиц в сфере киберкриминалистики. — Санкт-Петербург: Санкт-Петербургский Государственный Университет, 2019. — 34 с.

2 АНАЛИТИЧЕСКИЙ РАЗДЕЛ

2.1 Datagen. Top 10 Face Datasets for Facial Recognition and Analysis [Электронный ресурс]. — Режим доступа: <https://datagen.tech/blog/face-datasets/>

2.2 Github. DigiFace-1M Dataset [Электронный ресурс]. — Режим доступа: <https://github.com/microsoft/DigiFace1M>;

- 2.3 Wider Face: A Face Detection Benchmark [Электронный ресурс]. — Режим доступа: <http://shuoyang1213.me/WIDERFACE/>
- 2.4 Papers with code. WIDER (Web Image Dataset for Event Recognition) [Электронный ресурс]. — Режим доступа: <https://paperswithcode.com/dataset/wider>
- 2.5 Habr. OpenCV в Python [Электронный ресурс]. — Режим доступа: <https://habr.com/ru/articles/519454/>
- 2.6 Yichen Qian. Unsupervised Face Normalization with Extreme Pose and Expression in the Wild / Yichen Qian, Weihong Deng, Jiani Hu. – Computer Vision Foundation, 2019. – 8 с.
- 2.7 Tensorflow. Learn [Электронный ресурс]. — Режим доступа: <https://www.tensorflow.org/learn>
- 2.8 Hocktraining. Преимущества языка Python [Электронный ресурс]. — Режим доступа: <https://www.hocktraining.com/blog/preimuschestva-yazyka-python>
- 2.9 Habr. Распознаем лица на фото с помощью Python и OpenCV [Электронный ресурс]. — Режим доступа: <https://habr.com/ru/articles/301096/>
- 2.10 Xailient. Face Detector Comparison on a Resource-Constrained Device [Электронный ресурс]. — Режим доступа: <https://xailient.com/blog/face-detector-comparison-on-a-resource-constrained-device/>
- 2.11 Pyimagesearch. OpenCV Face Recognition [Электронный ресурс]. — Режим доступа: <https://pyimagesearch.com/2018/09/24/opencv-face-recognition/>
- 2.12 Github. Unsupervised Face Normalization with Extreme Pose and Expression in the Wild [Электронный ресурс]. — Режим доступа: <https://github.com/mx54039q/fnm>
- 2.13 Habr. Сравнение фреймворков для глубокого обучения [Электронный ресурс]. — Режим доступа: <https://habr.com/ru/companies/otus/articles/443874/>

3 ТЕХНОЛОГИЧЕСКИЙ РАЗДЕЛ

3.1 Github. MTCNN detection [Электронный ресурс]. — Режим доступа: https://github.com/kpzhang93/MTCNN_face_detection_alignment

3.2 Ntechlab. FindFace Multi [Электронный ресурс]. — Режим доступа: <https://docs.ntechlab.com/projects/ffmulti/en/2.1.3/index.html>

3.3 Kaggle. Guide to MTCNN in facenet-pytorch [Электронный ресурс]. — Режим доступа: <https://www.kaggle.com/code/timesler/guide-to-mtcnn-in-facenet-pytorch/notebook?scriptVersionId=26165086>

3.4 Habr. Классификатор изображений [Электронный ресурс]. — Режим доступа: <https://habr.com/ru/companies/dmlabs/articles/205842/>

3.5 Hugging Face. CLIP [Электронный ресурс]. — Режим доступа: https://huggingface.co/docs/transformers/model_doc/clip

3.6 Github. CLIP [Электронный ресурс]. — Режим доступа: <https://github.com/openai/CLIP>

3.7 Towards data science. How to train your CLIP [Электронный ресурс]. — Режим доступа: <https://towardsdatascience.com/how-to-train-your-clip-45a451dcd303>

3.8 Habr. Универсальный zero-shot классификатор на Android [Электронный ресурс]. — Режим доступа: <https://habr.com/ru/articles/546586/>

3.9 Pinecone. Zero-shot Image Classification with OpenAI's CLIP [Электронный ресурс]. — Режим доступа: <https://www.pinecone.io/learn/series/image-search/zero-shot-image-classification-clip/>

3.10 Medium. A Guide to Fine-Tuning CLIP Models with Custom Data [Электронный ресурс]. — Режим доступа: <https://medium.com/aimonks/a-guide-to-fine-tuning-clip-models-with-custom-data-6c7c0d1416fb>

3.11 Github. Simple image training using CLIP model [Электронный ресурс]. — Режим доступа: https://github.com/UKPLab/sentence-transformers/blob/master/examples/training/clip/train_clip.ipynb

3.12 Github. Deepfake detection challenge [Электронный ресурс]. — Режим доступа: <https://github.com/NTech-Lab/deepfake-detection-challenge>

ПРИЛОЖЕНИЯ

Приложение А — Графический материал

Приложение Б — Код извлечения изображений из видеофрагментов

Приложение В — Код вырезки кадров, включающих лица

Приложение Г — Код для предсказания с моделью

Приложение Д — Код для обучения (тюнинга) модели

Приложение А

Графический материал



МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение высшего образования

«МИРЭА – Российский технологический университет»

РТУ МИРЭА

Институт Информационных Технологий
Кафедра Прикладной математики

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА

Тема:

Разработка нейронной сети портретной классификации

-
- Студент: Черных А.Н.
 - Группа: ИМБО-01-20
 - Руководитель: к.т.н., доцент Ледовская Е.В.

Москва 2024

Рисунок А.1 — Первый слайд презентации



Постановка цели

Цель — разработка нейронной сети портретной классификации для различия рисованных и реальных лиц в условиях применения новых технических и технологических средств сбора, передачи, обработки и выдачи информации.

Задачи:

- Изучение методов анализа изображений;
- Разработка классификатора для различия рисованных и реальных лиц;
- Оценка результатов классификации.

Рисунок А.2 — Второй слайд презентации



Анализ изображений лиц

Анализ изображений лиц — это область исследований в компьютерном зрении, которая занимается распознаванием, классификацией и интерпретацией характеристик лиц на изображениях.



Рисунок 1 — Модель конвейера для обработки изображений лиц

Рисунок А.3 — Третий слайд презентации



Портретная классификация

Классификация — это область машинного обучения, которая специализируется на решении задач, связанных с присвоением объекта к определённой группе (классу), заданной изначально.

Рисунок А.4 — Четвёртый слайд презентации



Задачи в рамках разработки

1. Поиск видеофрагментов, включающих в себя рисованных персонажей, в открытом доступе.
2. Разбиение видео на кадры (изображения).
3. Детекция области лица на каждом изображении.
4. Сбор полученных изображений в датасет.
5. Обучение классификатора на полученном датасете.
6. Анализ работы классификатора.

Рисунок А.5 — Пятый слайд презентации

Wider Face

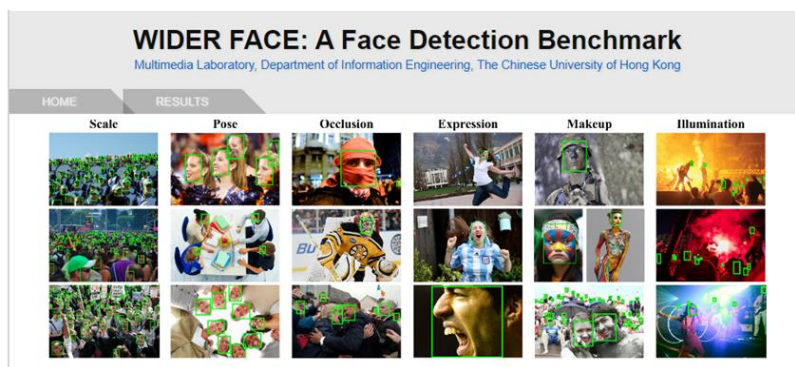


Рисунок 2 — Официальный сайт WiderFace

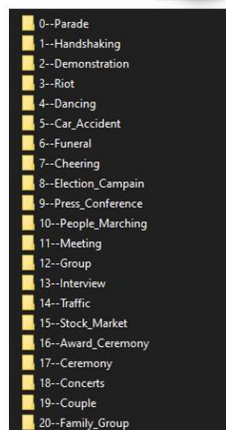


Рисунок 3 — Категории изображений WiderFace

Рисунок А.6 — Шестой слайд презентации

Видеофрагменты

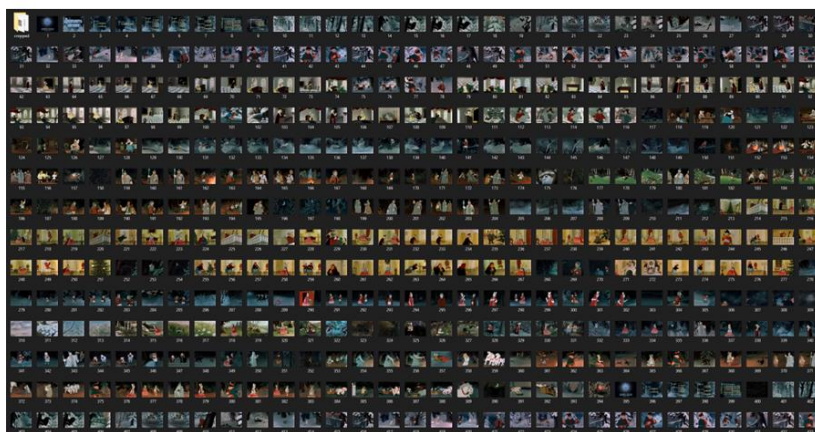


Рисунок 4 — Выделенные из видеофрагмента кадры

Рисунок А.7 — Седьмой слайд презентации

Детекция лиц

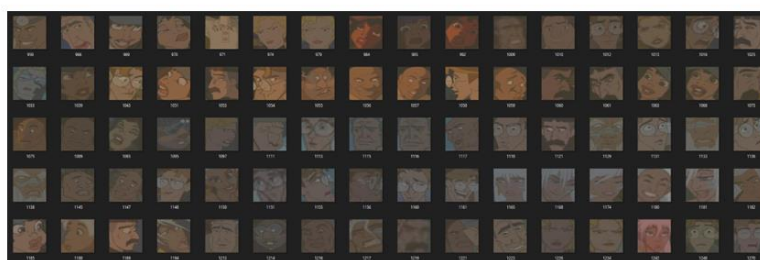


Рисунок 5 — Фрагменты с определёнными лицами

Рисунок А.8 — Восьмой слайд презентации

Собранный датасет



```
DatasetDict({
  train: Dataset({
    features: ['image', 'label'],
    num_rows: 3817
  })
  test: Dataset({
    features: ['image', 'label'],
    num_rows: 1908
  })
})
```

Рисунок 6 — Информация о собранном датасете

Рисунок А.9 — Девятый слайд презентации

Модель CLIP

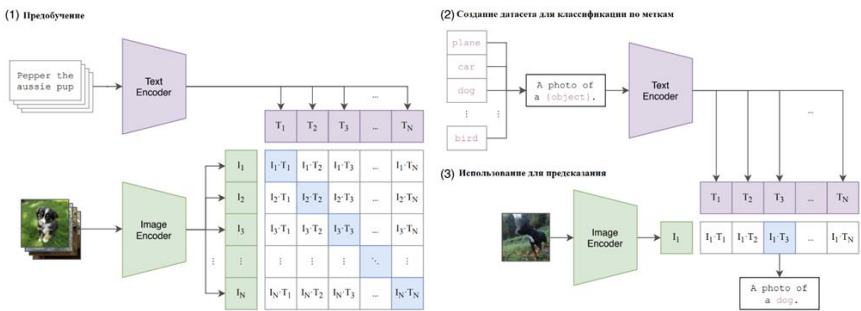


Рисунок 7 — Принцип работы модели CLIP

Модель показала точность классификации ~70%

Рисунок А.10 — Десятый слайд презентации

Обучение

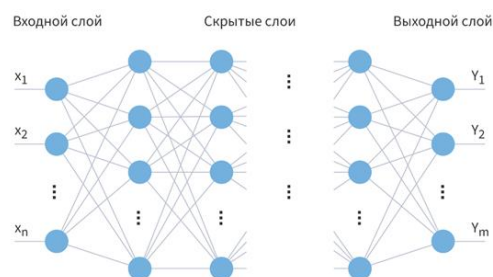


Рисунок 8 — Схематичное представление нейронной сети

Обучение занимает 30 эпох. После обучения модель показала точность классификации ~80%.

Рисунок А.11 — Одиннадцатый слайд презентации

Обучение

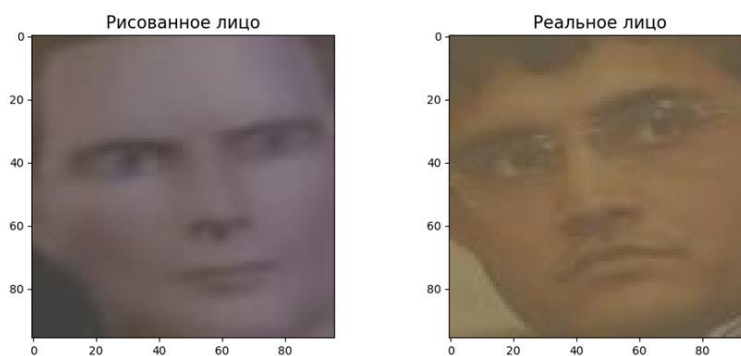


Рисунок 9 — Пример работы классификатора

Рисунок А.12 — Двенадцатый слайд презентации

Анализ результатов



```
precision = 0.7313278008298755  
recall = 0.8576642335766423  
F-1 = 0.7894736842105263
```

Рисунок 11 — Значение метрик

Рисунок А.13 — Тринадцатый слайд презентации

Анализ результатов

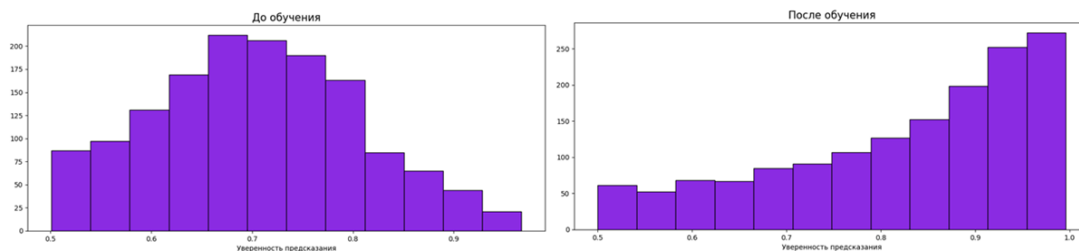


Рисунок 10 — Распределение уверенности модели до и после обучения

Рисунок А.14 — Четырнадцатый слайд презентации

Вывод



В результате данной выпускной квалификационной работы была **рассмотрена область анализа изображений лиц**, а также проведён **обзор существующих решений** в обозначенной области.

В процессе анализа была **определена методика сбора датасета** и **обоснован выбор инструментальных средств разработки**.

Далее была **разработана нейронная сеть** классификации для различия реальных и рисованных лиц.

Рисунок А.15 — Пятнадцатый слайд презентации



Спасибо за
внимание

Рисунок А.16 — Шестнадцатый слайд презентации

Приложение Б

Код извлечения изображений из видеофрагментов

```
import time
import cv2
import os

cap = cv2.VideoCapture("dir") #dir - dir where the video lies
t = time.time()

i = 0
os.chdir("dir") #dir - dir to save frames
while (True):
    ret, frame = cap.read()
    if time.time() - t >= 0.1:
        t = time.time()
        i += 1
        cv2.imwrite(str(i) + ".jpg", frame)
        print(f'frame {i} saved')
```

Приложение В

Код вырезки кадров, включающих лица

```
from facenet_pytorch import MTCNN, InceptionResnetV1
from PIL import Image
import os
import torchvision.transforms as T
import numpy as np
import torch

def _inverse_norm(images):
    if isinstance(images, torch.Tensor):
        images = images.cpu().permute(1, 2, 0).numpy()
        NORM_MEAN = np.array([0.485, 0.456, 0.406])
        NORM_STD = np.array([0.229, 0.224, 0.225])
        images = (images * NORM_STD[None, None]) + NORM_MEAN[None, None]
        images = np.clip(images, a_min=0.0, a_max=1.0)
    return images

mtcnn = MTCNN(image_size=96)
resnet = InceptionResnetV1(pretrained='vggface2').eval()
for i in range(1049):
    os.chdir('dir') #dir – dir where the images lies
    img = Image.open(str(i+1)+' .jpg')
    img_cropped = mtcnn(img)
    if img_cropped!=None:
        img_cropped=_inverse_norm(img_cropped)
        img = T.ToPILImage(img_cropped)
        os.chdir('dir') #dir – dir where the cropped images will be saved
        img.save(str(i+1)+' .jpg')
```

Приложение Г

Код для предсказания с моделью

```
dataset = load_dataset(path="dir") #dir – dir where the dataset lies
test = dataset['test']
labels = test.info.features['label'].names
clip_labels = [f"a photo of a {label}" for label in labels]
# Choose computation device
device = "cuda:0" if torch.cuda.is_available() else "cpu"

# Load pre-trained CLIP model
model, preprocess = clip.load("ViT-B/32", device=device, jit=False)
# Move the model to the device
model_id = "openai/clip-vit-base-patch32"

preds = []
batch_size = 15
all_probs = []
for i in tqdm(range(0, len(test), batch_size)):
    i_end = min(i + batch_size, len(test))
    j=i
    while (j<i_end):
        image = preprocess(test['image'][j]).unsqueeze(0).to(device)
        print(f"image {j} processed")
        text = clip.tokenize(clip_labels).to(device)
        with torch.no_grad():
            logits_per_image, logits_per_text = model(image, text)
            probs = logits_per_image.softmax(dim=-1).cpu().numpy()
            preds.extend(np.argmax(probs, axis=1))
            all_probs.append(max(probs[0]))
        j+=1

true_preds = []
true_probs = []
for i, label in enumerate(test['label']):
    if label == preds[i]:
        true_preds.append(1)
        true_probs.append(all_probs[i])
    else:
        true_preds.append(0)
print(sum(true_preds) / len(true_preds))
print(true_probs)
```

Приложение Д

Код для обучения (тюнинга) модели

```
# Define a custom dataset
class image_title_dataset():
    def __init__(self, list_image, list_txt):
        # Initialize image paths and corresponding texts
        self.image = list_image
        # Tokenize text using CLIP's tokenizer
        self.title = clip.tokenize(list_txt)

    def __len__(self):
        return len(self.title)

    def __getitem__(self, idx):
        # Preprocess image using CLIP's preprocessing function
        image = preprocess(self.image[idx])
        title = self.title[idx]
        return image, title

dataset = load_dataset(path="dir") #dir – dir where the dataset lies
train = dataset['train']

# Choose computation device
device = "cuda:0" if torch.cuda.is_available() else "cpu"
# Load pre-trained CLIP model
model, preprocess = clip.load("ViT-B/32", device=device, jit=False)
list_image = []
list_txt = []
print(f'number of rows: {len(train)}')
i=0
for item in train:
    print(f'row {i} is processing')
    caption = item['label']
    if caption == 0:
        caption = 'a photo of a character'
    else: caption = 'a photo of a human'
    list_image.append(item['image'])
    list_txt.append(caption)
    i+=1
dataset = image_title_dataset(list_image, list_txt)
# Define a custom dataset
train_dataloader = DataLoader(dataset, batch_size=20, shuffle=True) #Define your
own dataloader
# Function to convert model's parameters to FP32 format
def convert_models_to_fp32(model):
```

```

        for p in model.parameters():
            p.data = p.data.float()
            p.grad.data = p.grad.data.float()
    if device == "cpu":
        model.float()
    # Prepare the optimizer
    optimizer = torch.optim.Adam(model.parameters(), lr=5e-5, betas=(0.9, 0.98),
eps=1e-6,
                                weight_decay=0.2) # the lr is smaller, more safe for fine tuning to new
dataset
    # Specify the loss function
    loss_img = nn.CrossEntropyLoss()
    loss_txt = nn.CrossEntropyLoss()
    # Train the model
    num_epochs = 30
    for epoch in range(num_epochs):
        pbar = tqdm(train_dataloader, total=len(train_dataloader))
        for batch in pbar:
            optimizer.zero_grad()
            images, texts = batch
            images = images.to(device)
            texts = texts.to(device)
            # Forward pass
            logits_per_image, logits_per_text = model(images, texts)
            # Compute loss
            ground_truth = torch.arange(len(images), dtype=torch.long, device=device)
            total_loss = (loss_img(logits_per_image, ground_truth) +
loss_txt(logits_per_text, ground_truth)) / 2
            # Backward pass
            total_loss.backward()
            if device == "cpu":
                optimizer.step()
            else:
                convert_models_to_fp32(model)
                optimizer.step()
                clip.model.convert_weights(model)
            pbar.set_description(f"Epoch {epoch+1}/{num_epochs}, Loss:
{total_loss.item():.4f}")
            if ((epoch+1) % 3) == 0:
                torch.save({
                    'model_state_dict': model.state_dict(),
                    'optimizer_state_dict': optimizer.state_dict()
                }, f"dir\model_{epoch+1}_{total_loss}.pt") # dir – dir where model files will be
saved

```